Infinite Descent (CryptoHack) – WriteUp

Debonmoy Pal

Name – Infinite Descent

Description - Finding large primes is slow, so I've devised an optimisation.

Code Analysis:

descent.py

```python
#!/usr/bin/env python3

import random
from Crypto.Util.number import bytes_to_long, isPrime

FLAG = b"crypto{????????????????????}"


def getPrimes(bitsize):
    r = random.getrandbits(bitsize)
    p, q = r, r
    while not isPrime(p):
        p += random.getrandbits(bitsize//4)
    while not isPrime(q):
        q += random.getrandbits(bitsize//8)
    return p, q


m = bytes_to_long(FLAG)
p, q = getPrimes(2048)
n = p * q
e = 0x10001
c = pow(m, e, n)

print(f"n = {n}")
print(f"e = {e}")
print(f"c = {c}")

```

From the code I can understand, both p and q begin with same value r. The code then increases them independently until each becomes prime. Since both prime numbers originate from the same 2048-bit number, their difference is very small.

This is the vulnerability that needs to be exploited.

Exploitaion:

This challenge can be solved by Fermat's Factorization because:
RSA generates a modulus using: n=pq

Fermat's factorization uses the identity: n = a^2 – b^2 = (a-b) (a+b)

So, I need to find int a and b such that: a^2 – n = b^2

I can now recover the 2 factors: p=a-b, q=a+b

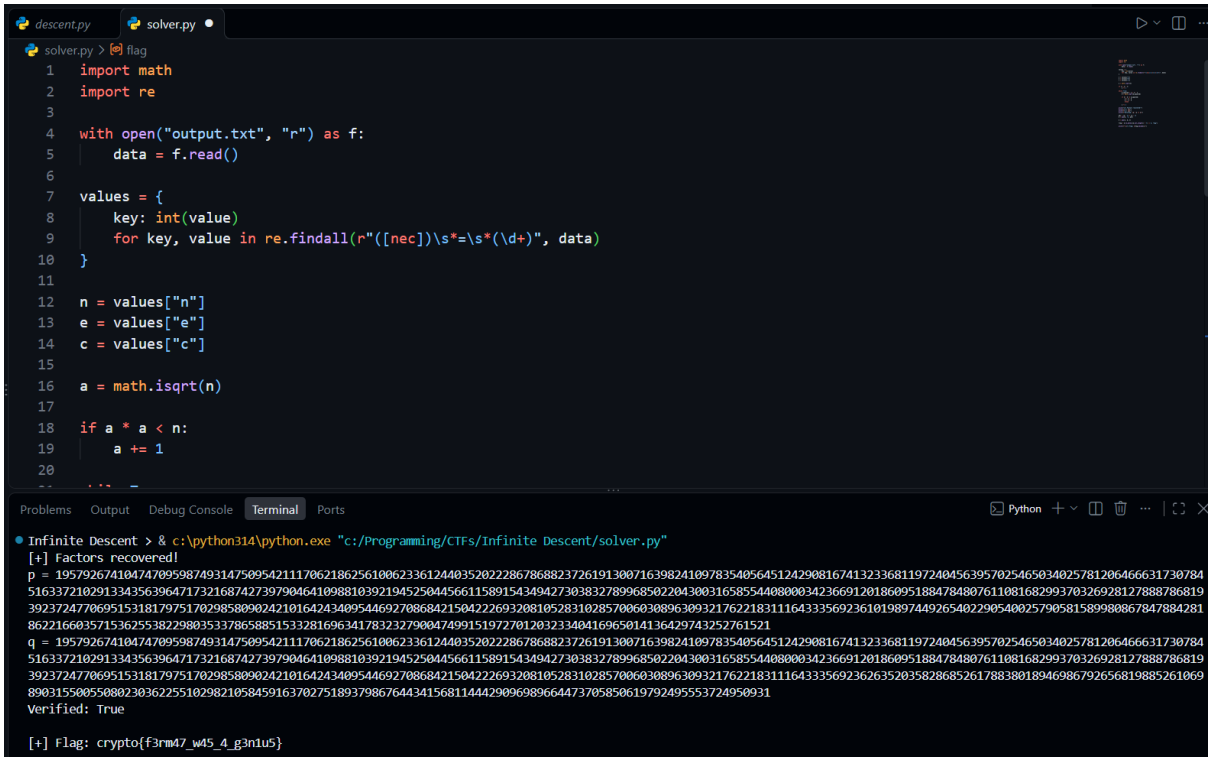
Since, p and q are too close, the initial value of a already satisfies the condition and so Fermat's factorization succeeds on the very first attempt.

Flag recovery:

Output.txt contains RSA modulus n, public exponent e and ciphertext c.

I need to factor n to calculate the private exponent and then decrypt the ciphertext.

To do this I use the following solver script:



Final Flag: crypto{f3rm47_w45_4_g3n1u5}